

MSE 352: Digital Logic and Microcontrollers

Final Project Report

Ishaan Dhadral - 301372251

Abhimanyu Khanna - 391556250

Tom Dang - 301548582

December 5, 2024

SCHOOL OF MECHATRONIC SYSTEMS ENGINEERING
SIMON FRASER UNIVERSITY

Introduction

In this project, we will design and implement a fan speed control system using the 8051 microcontroller, focusing on precise real-time regulation through Pulse Width Modulation (PWM) and feedback control. The core tasks include measuring the fan's RPM using timer interrupts to count the falling edges of the PWM signal, generating the PWM signal with adjustable duty cycles via the 8051's timers, and displaying the measured RPM on 7-segment displays using multiplexing techniques for efficient output. Additionally, a closed-loop control system will be implemented to dynamically adjust the duty cycle based on real-time RPM feedback, maintaining the fan speed within a 5% tolerance of the target value. This project demonstrates the integration of digital logic, timer operations, and real-time control, showcasing the practical application of microcontroller-based automation in mechatronics.

System Design Overview

The hardware system includes the following components:

- Microcontroller: 8051 MCU (40 pins) for control and data processing.
- Fan: A DC brushless fan with a maximum RPM of 2400 at 100% duty cycle.
- Display: Four 7-segment LEDs for displaying the RPM.
- MOSFET: Used for PWM signal control of the fan.
- BJTs: Control the multiplexing of the 7-segment displays.
- DIP Switch: An 8-bit switch for manually adjusting the duty cycle.

Circuit Overview:

- The microcontroller generates a PWM signal that controls the fan speed via a MOSFET.
- BJTs handle display multiplexing to minimize power consumption.
- The DIP switch allows manual adjustment of the PWM duty cycle.

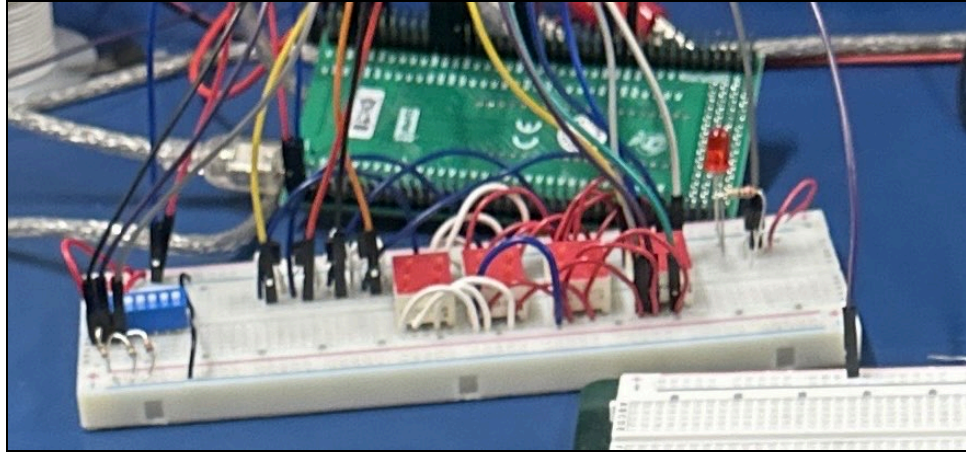


Figure 1: Overview of Circuit layout

Code Setup:

The code is a single standalone C file, with 4 sections: headers, global variables, prototypes, and main loop control. There are 2 headers used for this project: the `STARTUP.A51` and the Standard Input Output headers. The global variables declare the inputs and outputs from the microcontroller, fixed arrays for displaying numbers, and variables for RPM control. The prototypes include a function to calculate RPM, a function to decode the PWM signal from the DIP switch, display 4-digit RPM, and set up the timer configuration.

The main loop first initialized the timer configuration. Then, it enters a periodic cycle. The cycle starts with reading in the input from the DIP switch to get the on-time duty cycle and the ideal RPM, display the actual RPM of the fan, and then adjust the RPM closer to the ideal RPM. Lastly, the program does a self-check to keep on time duty cycle stay within 0 and 1. The cycle repeats itself.

```

// Main loop
void main(void) {
    setup_timers(); // Initialize Timer 0 and Timer 1

    while (1) {
        // Step 1: Read target RPM from DIP switch
        target_RPM = On_time_duty_cycle() * 240; // Scale DIP switch value to RPM (max 2400 RPM)

        // Step 2: Calculate RPM based on edge count
        current_RPM = calculate_RPM();
        global_display_num = current_RPM; // Update number to display on 7-segment

        // Step 3: Adjust PWM duty cycle based on target RPM
        if (current_RPM < target_RPM - (target_RPM * 0.05)) { // If current RPM is 5% below target
            PWM_on_time += 1; // Increase duty cycle (step size = 1 us)
        } else if (current_RPM > target_RPM + (target_RPM * 0.05)) { // If current RPM is 5% above target
            if (PWM_on_time > 1) { // Ensure on-time does not go below 0
                PWM_on_time -= 1; // Decrease duty cycle (step size = 1 us)
            }
        }

        // Ensure PWM_on_time stays within bounds
        if (PWM_on_time > PWM_period) PWM_on_time = PWM_period; // Max is 100%
        if (PWM_on_time < 1) PWM_on_time = 1; // Min is ~2% duty cycle
    }
}

```

Figure 2: Main .c setup

Dip Switch Control:

The fan control system incorporates an 8-bit DIP switch to enable manual selection of the PWM duty cycle, providing flexibility in controlling the fan's speed. Of these, three pins—**RA0**, **RA1**, and **RA2**—represent the most significant bits and allow the user to select a duty cycle percentage ranging from 0% to 100%. The `On_time_duty_cycle()` function in the control header decodes the state of these bits into discrete duty cycle levels, with examples such as **RA0=1, RA1=1, RA2=1** mapping to a 100% duty cycle and **RA0=0, RA1=0, RA2=0** mapping to 0%. This implementation offers a simple yet effective method for defining the desired speed of the fan, which is essential for dynamic testing and validating the control system's performance.

PWM Control:

The fan's speed is regulated through Pulse Width Modulation (PWM), where the power supplied to the fan is controlled by modulating the on-to-off time ratio of the output signal. The system generates the PWM signal via the `PWM_pin` and updates the duty cycle dynamically based on feedback from the tachometer. In the software implementation, the `pwm_output()` function toggles the `P3` port high for the on-time and low for the off-time, with timing handled using `timer0_delay_100us()` for software-based delays. This functionality is also managed in the `timer1_ISR()` interrupt, ensuring precise and seamless PWM signal generation. This dual approach ensures consistent PWM signals and allows the duty cycle to be adjusted in real time to maintain the target fan speed.

```
// System inputs and outputs
sbit TACH_PIN = P0^4;    // Tachometer input pin
sbit RA0 = P0^0;        // DIP switch bit 0
sbit RA1 = P0^1;        // DIP switch bit 1
sbit RA2 = P0^2;        // DIP switch bit 2
sbit PWM_pin = P3^0;    // PWM output pin

// Global variables
unsigned char digits[] = { // Array to store segment patterns for numbers 0-9
    0xC0, // HEX for 0
    0xF9, // HEX for 1
    0xA4, // HEX for 2
    0xB0, // HEX for 3
    0x99, // HEX for 4
    0x92, // HEX for 5
    0x82, // HEX for 6
    0xF8, // HEX for 7
    0x80, // HEX for 8
    0x90, // HEX for 9
    0xFF // HEX for [Blank]
};

volatile unsigned int edge_count = 0;    // Falling edge count for tachometer
volatile unsigned int global_display_num = 0; // Number to display on 7-segment
unsigned int target_RPM = 0;            // Target RPM from DIP switch
unsigned int current_RPM = 0;           // Measured RPM
unsigned int PWM_on_time = 50;          // On time in us (initially 100% duty cycle)
unsigned int PWM_period = 50;           // PWM period in us (20 kHz frequency)

// Function prototypes
unsigned int calculate_RPM();            // Calculate RPM based on edge count
int On_time_duty_cycle();               // Get duty cycle setting from DIP switch
void display_num(unsigned int num);     // Display number on 7-segment
void setup_timers();                   // Configure and start timers
```

Figure 3: Dip Switch input code

Tachometer Reading:

Speed The tachometer functionality measures the fan's RPM by counting falling edges of the output signal generated by the fan motor. Timer 0 is configured in Mode 2 to operate as an 8-bit auto-reload counter, capturing the edges within each PWM period. The RPM calculation is carried out using the formula

$RPM = \frac{edges \times 60}{PWM \text{ Period [s]} \times 2}$, implemented in the `calculate_RPM()` function. This method ensures accurate RPM measurement without interference from other system components. However, initial attempts using delays for timing introduced inconsistencies and inaccuracies in the RPM calculation, as described below.

```

unsigned int read_fan_rpm() {
    unsigned int time_ticks = measure_time_between_rising_edges();
    return calculate_rpm(time_ticks);
}

unsigned int get_RPM(int on_time){
    // 0 RPM with 0% duty cycle and 2400 RPM with 100% duty cycle
    // 240 RPM gain for every 10% gain
    return 240 * on_time;
}

unsigned int get_RPM_time_pulse() {
    unsigned int start_time, end_time, tick_count;

    // Wait for the first rising edge of the pulse
    while (time_pulse == 0); // Wait for time_pulse to go HIGH
    start_time = timer0_read(); // Capture start time

    // Wait for the next rising edge of the pulse
    while (time_pulse == 1); // Wait for time_pulse to go LOW
    while (time_pulse == 0); // Wait for time_pulse to go HIGH again
    end_time = timer0_read(); // Capture end time

    // Calculate the tick count
    if (end_time >= start_time) {
        tick_count = end_time - start_time; // No overflow
    } else {
        tick_count = (65536 - start_time) + end_time; // Account for overflow
    }

    // Ensure no division by zero
    if (tick_count == 0) {
        return 0; // Return 0 RPM if the pulse interval is invalid
    }
    return 60000000 / (1.2 * tick_count);
}

```

Figure 4: Tachometer read function with delays

Results and Discussion:

During development, using delays such as `timer0_delay_100us()` for timing operations caused significant disruptions in the system's functionality. Since delays temporarily blocked the execution of other tasks, this led to conflicting timer operations, particularly between the tachometer edge counting and the PWM signal generation. As a result, RPM readings were often inaccurate due to missed tachometer edges, and the PWM signal exhibited irregularities, compromising precise fan speed control. The issue can be further resolved by transitioning from delays to counters for timing. By leveraging the counters in interrupt-driven

processes, the tachometer's edge counting and the PWM signal generation were handled seamlessly without interfering with each other, resulting in a stable and accurate control system. If our time constraints were not as tight, we believe our group would have been able to implement a working system and more memory-efficient code.

```
void timer0_delay_100us() {  
    TMOD = 0x01; // Timer 0, Mode 1 (16-bit timer)  
    TH0 = 0xFF;  // Load high byte of the preload value  
    TL0 = 0xAC;  // Load low byte of the preload value  
    TR0 = 1;     // Start Timer 0  
    while (TF0 == 0); // Wait for the timer to overflow  
    TR0 = 0;     // Stop Timer 0  
    TF0 = 0;     // Clear the Timer 0 overflow flag  
}
```

Figure 5: Delay code

RPM readings were recorded at duty cycles ranging from 10% to 100%, with an expected speed increment of 240 RPM per 10% increase in duty cycle, up to the maximum fan speed of 2400 RPM. The results demonstrated that the system consistently maintained the fan speed within 5% of the target RPM across all tested duty cycles. Taking into account that the duty cycle was measurable on the oscilloscope, however, the period when taking the delays into account was quite long in the range of 200-500 ms. However, the stability of our code and the accurate integration of PWM generation and tachometer feedback are even due to the delay discrepancy.

Conclusion:

The development of the fan speed control system using the 8051 microcontroller provided valuable insights into the practical implementation of digital logic, real-time control, and embedded systems design. Through this project, we explored the intricacies of Pulse Width Modulation (PWM) signal generation, the use of feedback mechanisms for closed-loop control, and the challenges of interfacing with hardware components like tachometers and DIP switches.

Additionally, the project emphasized the importance of modular and structured coding practices. By leveraging reusable functions and segregating responsibilities across header files, we achieved a streamlined and maintainable codebase. This approach facilitated the seamless integration of components like the display, DIP switch, and control logic while maintaining system flexibility.

Finally, the experience of testing and refining the system underlined the necessity of iterative development and thorough validation. Through systematic testing at various duty cycles, we were able to achieve a robust and accurate control system that consistently met its design objectives. The knowledge gained from debugging and optimizing the system will undoubtedly inform future projects in embedded system design and real-time control.